

The CPU

- The Central Processing Unit (**CPU**) is "the computer": the component that executes program instructions
- In addition to circuitry to execute instructions, the CPU has a set of *registers*
- Registers, presented next, are a *very fast* form of memory

Registers

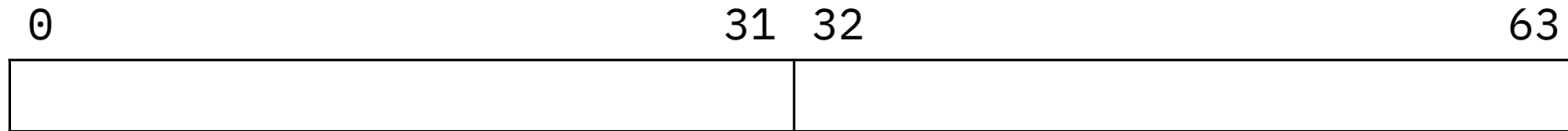
- The CPU registers are:
 - The Program Status Word, or **PSW**, to control program execution
 - **General registers**, for arithmetic and logical operations, and to hold addresses of data and instructions
 - Also (not covered in this course):
 - Floating-Point Registers for floating-point arithmetic and data
 - Control Registers, used by the hardware and the operating system
 - Access Registers, Vector Registers
- We'll start with the General registers, then we'll look at the PSW

General Registers

- There are sixteen General registers, each identified by a number, **0** to **15** (**0** to **F** in hexadecimal)
- Instead of saying “**Register x**”, we will refer to the general registers using the notation: **R0**, **R1... R15**
- Each General register is **64 bits** in size (picture next)

General Registers

- Graphically:



- When we discuss 32- and 64-bit arithmetic, we'll understand why this picture shows two 32-bit parts
- What are registers for?

General Registers

- General registers are mainly used for two things:

1. Arithmetic and logical operations

Registers can contain **binary integers** for arithmetic calculations or **bit strings** for logical operations (we'll cover both in detail later in the course)

2. Holding addresses of data and instructions

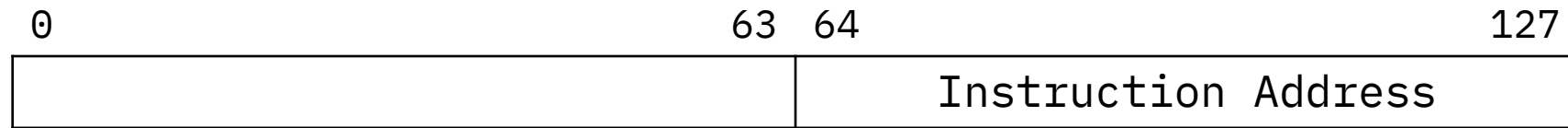
We'll soon see how registers are used for addressing data and instructions in memory

The PSW

- Usually, the Program Status Word (PSW) is of no immediate concern, and you need not worry about its contents. It is an internal register that contains fields indicating the status of the program being executed
- For our purposes, we need to know only a few parts of a PSW:
 - The Instruction Address (IA)
 - The Condition Code (CC)
 - The Program Mask (PM)
- Of these, the IA is the most important now; we'll see more about the others later

The PSW - Instruction Address

- The PSW is a 128-bit register
- The last 64 bits of the PSW are the Instruction Address (IA):



- The PSW IA **always has the address of the next instruction** to be executed (not the one being executed)
- *This is what makes the contents of a memory location an instruction*

Program execution

- This is how the CPU executes instructions:
 - Step 1. The memory address of the first instruction to be executed is placed in the PSW

Program execution

- This is how the CPU executes instructions:
 - Step 1. The memory address of the first instruction to be executed is placed in the PSW
 - Step 2. The instruction pointed to by the PSW is retrieved from memory by the CPU

Program execution

- This is how the CPU executes instructions:
 - Step 1. The memory address of the first instruction to be executed is placed in the PSW
 - Step 2. The instruction pointed to by the PSW is retrieved from memory by the CPU
 - Step 3. The PSW is updated to point to the next instruction in memory

Program execution

- This is how the CPU executes instructions:
 - Step 1. The memory address of the first instruction to be executed is placed in the PSW
 - Step 2. The instruction pointed to by the PSW is retrieved from memory by the CPU
 - Step 3. The PSW is updated to point to the next instruction in memory
 - Step 4. The retrieved instruction is executed

Program execution - the next instruction

- If the retrieved instruction did not cause a Branch (“Go To”), go back to Step 2
 - Note that the PSW is already pointing to the the next instruction
- Otherwise, put the memory address to be branched to in the PSW, then go back to Step 2
- This leaves many questions unanswered (How was the program loaded in memory? How does it stop?) but provides the basic ideas
- What do instructions look like?

Instructions - Operation code

- Instructions, in memory, look like this:

Operation code	Operands
----------------	----------

- *Note this and other examples that follow are a simplification; we will learn the actual instruction formats later*
- The Operation code, as the name implies, is the operation to be performed
 - For example: Branch (“Go to”), Add, Multiply...
 - It is often abbreviated “Opcode”

Instructions - Operands

- The Operands are what the operation acts on. For example:
 - In the instruction “Branch to Calculate_Tax”, the instruction at “Calculate_Tax” is the operand
 - In the instruction “Add Tax to Register 3”, the bytes at a field called “Tax” and General register 3 are the operands
 - In the instruction “Subtract Debit from Balance”, the bytes at the fields called “Debit” and “Balance” are the operands

Instructions - Operands

- This is how our instructions would look in memory
 - Remember these are not the exact instruction formats, only approximate representations to illustrate the mechanism

- "Branch to Calc_Tax":

Branch	Address of Calc_Tax
--------	---------------------

- "Add Tax to Register 3":

Add	3	Address of Tax
-----	---	----------------

- "Subtract Debit from Balance":

Subtract	Address of Balance	Address of Debit
----------	--------------------	------------------

Operand addressing

- How can we encode an operand's address?
- One possibility is that an operand's address is just that, its actual address in memory
- The problem with this approach is that z/Architecture addresses are 64 bits (8 bytes) long. Instructions will be too big!

Opcode	8-byte operand 1 address	8-byte operand 2 address
--------	--------------------------	--------------------------

- Actual addresses were an option when computer memories were small (in the 1950s!)

Operand addressing

- Actual memory addresses have another problem: *program relocation*. Consider a program loaded in memory and how its instructions would be encoded:

Address	Contents
2A000	(Program starts here)
...	Branch to Calc_Tax
...	
2A080	Calc_Tax: Add Tax to R3
...	
2A0C0	Tax: ...
...	



Opcode	Operand (Calc_Tax)
Branch	00000000000000002A080



Opcode	Operand (R3)	Operand (Tax)
Add	3	00000000000000002A0C0

Operand addressing

- Programs are not always loaded at the same address. When the program is loaded at a different location, every operand in memory must be adjusted:

Address	Contents
78000	(Program starts here)
...	Branch to Calc_Tax
...	
78080	Calc_Tax: Add Tax to R3
...	
780C0	Tax: ...
...	

Opcode	Operand (Calc_Tax)
Branch	000000000000 78080

Opcode	Operand (R3)	Operand (Tax)
Add	3	000000000000 780C0

Operand addressing

- When using actual memory addresses, every time a program is loaded, its operand addresses must be adjusted. That's too much work!
- The solution? To use offsets. Offsets (distances) from the program's starting location don't change:

Address	Offset	Contents
2A000	000	(Program starts here)
...		Branch to Calc_Tax
...		
2A080	080	Calc_Tax: Add Tax to R3
...		
2A0C0	0C0	Tax: ...
...		

Address	Offset	Contents
78000	000	(Program starts here)
...		Branch to Calc_Tax
...		
78080	080	Calc_Tax: Add Tax to R3
...		
780C0	0C0	Tax: ...
...		

Operand addressing

- z/Architecture computers use offsets for addressing operands
- The mechanism is called **base-displacement**
- It consists of using a register to point to an address (called the **base**), and use offsets (displacements) from the base
- Base-displacement is presented next